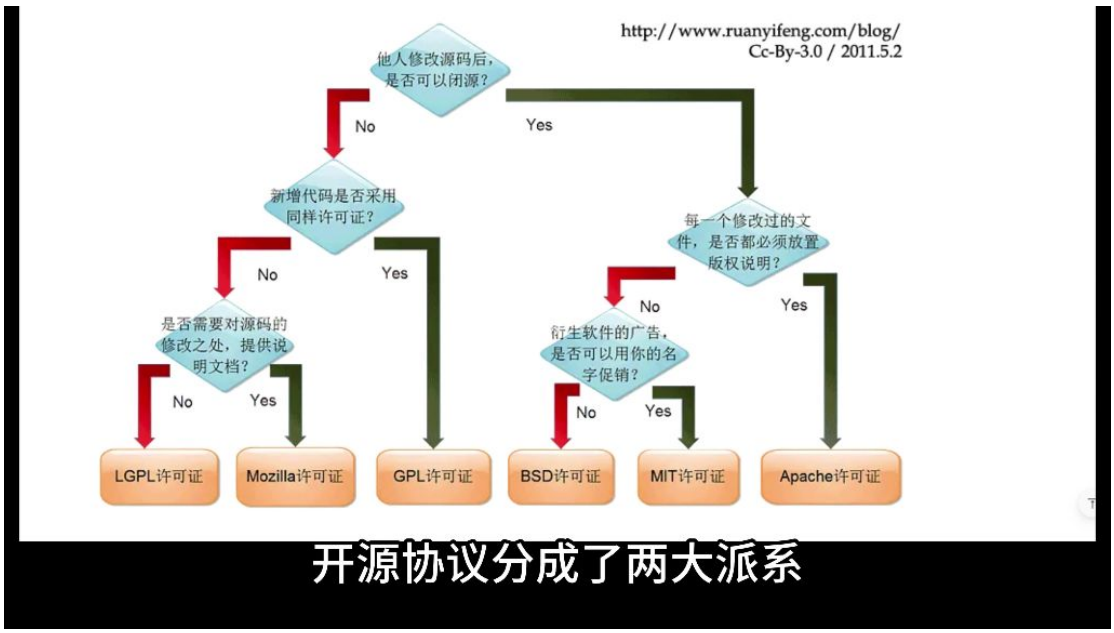
-- 以下为AI生成的图文笔记内容 --

## *一、创建自己的第一个仓库**00:00***

1. 选择开源协议 00:03

-


- **创建入口**：在GitHub网页右上角点击加号，选择"New repository"
- **开源协议作用**：规定他人对代码的使用权限和限制条件



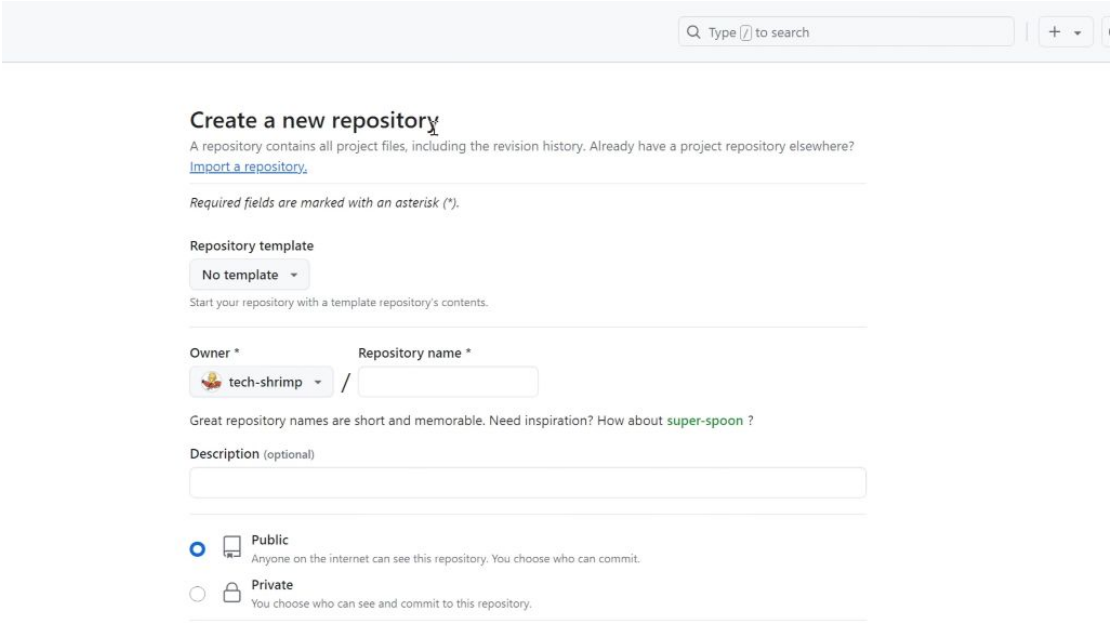

- **GPL系特点**：
  - **传染性**：修改后的衍生作品必须保持开源
  - **代表项目**：Linux、WordPress、Git等

- **争议点**：限制企业使用场景，接受度较低

- **宽松协议特点**：
  - **BSD协议**：需保留协议副本，禁止用作者名促销
  - **MIT协议**：只需保留协议副本，使用最自由
  - **Apache协议**：需为每个修改文件添加版权说明

2. 创建仓库 00:15

1）填写仓库名字

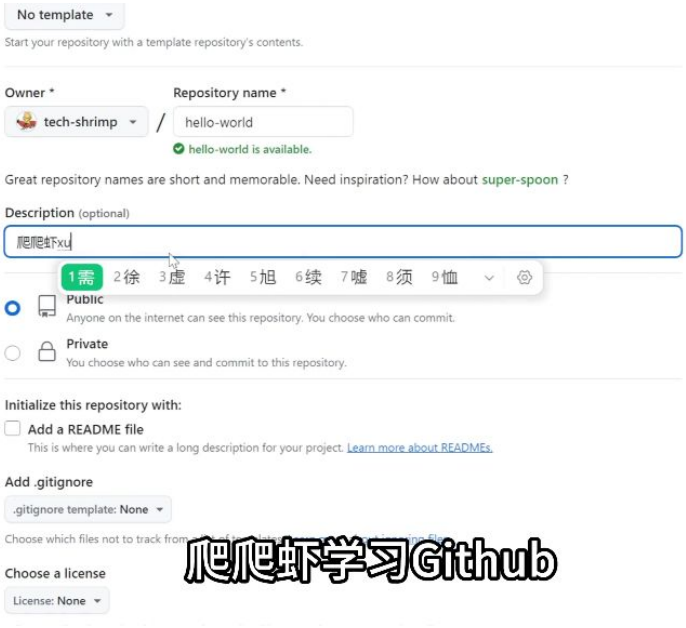



- **命名建议**：简短易记，如"hello-world"
- **描述字段**：可选填写项目简介
- **模板选择**：可从模板仓库初始化内容

2）仓库权限 00:35

- **Public**：互联网可见，可设置提交权限
- **Private**：仅指定人员可见和提交
- **历史变化**：GitHub现已免费提供私有仓库功能

3）添加文件 00:52

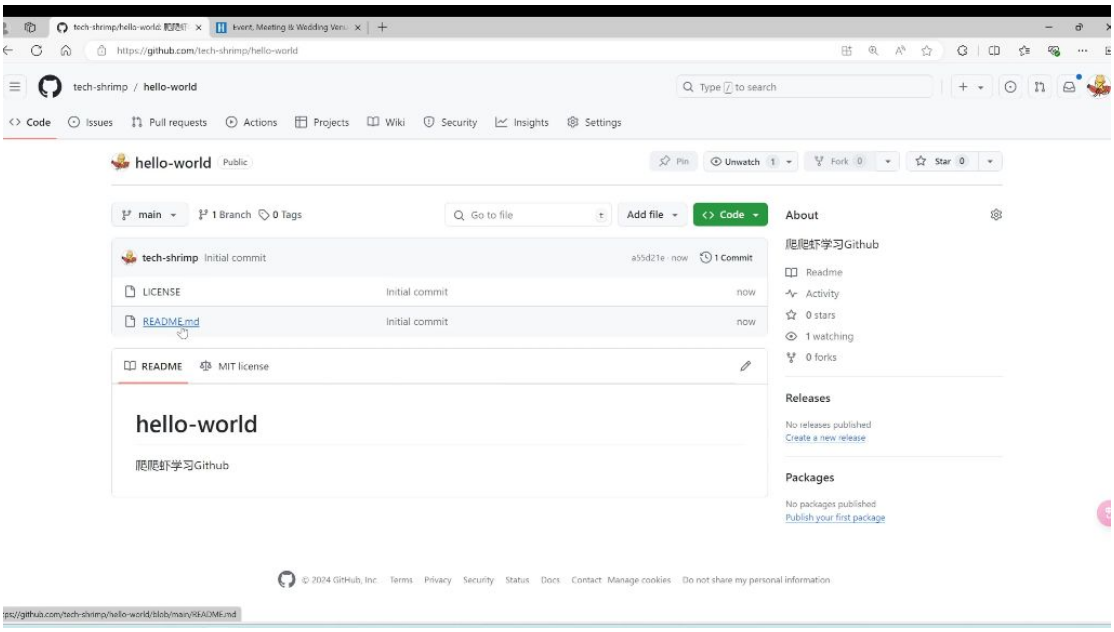



- 
- **README文件**：
  - **作用**：项目自述文件，说明项目用途、搭建和运行方法
  - **格式**：使用Markdown语法编写
- **.gitignore**：指定不跟踪的文件类型（后续课程详解）

4）开源协议 01:13

- GPL系 01:22
  - **LGPL**：宽松版GPL，新增代码不必使用原协议
  - **传染机制**：使用部分GPL代码将导致整个项目受GPL约束
- 比较宽松的开源协议 02:22
  - **选择建议**：根据项目需求选择，MIT协议适合大多数情况
  - **协议对比**：从严格到宽松依次为GPL→LGPL→Apache→MIT→BSD

5）创建仓库成功 02:49

- 

- **初始文件**：自动生成README和LICENSE文件
- **界面功能**：可通过网页直接编辑文件

6）编辑文件 03:06

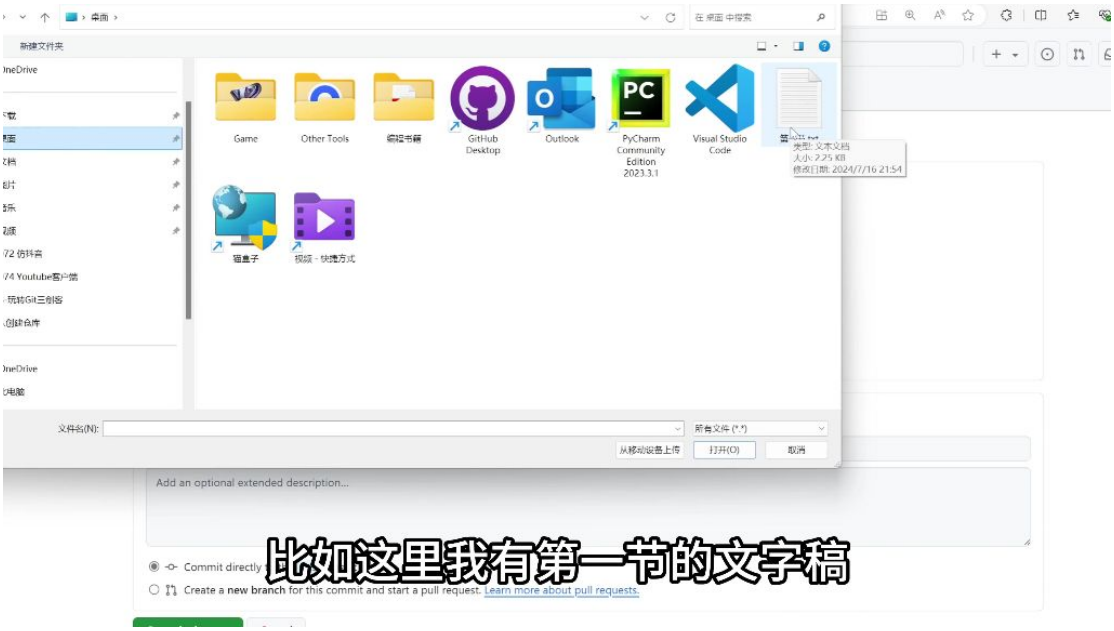
-

- **编辑入口**：文件右上角铅笔图标
- **预览功能**：可实时查看Markdown渲染效果
- **提交变更**：需填写变更说明（commit message）

7）commit概念 03:28

- 
- **本质**：仓库状态快照，记录所有文件状态
- **历史链路**：形成可回溯的修改记录
- **ID特性**：
  - 分为长ID和短ID，实际使用中短ID更方便
  - 通过哈希算法生成，保证唯一性
  - 别名：SHA值、提交ID、哈希值

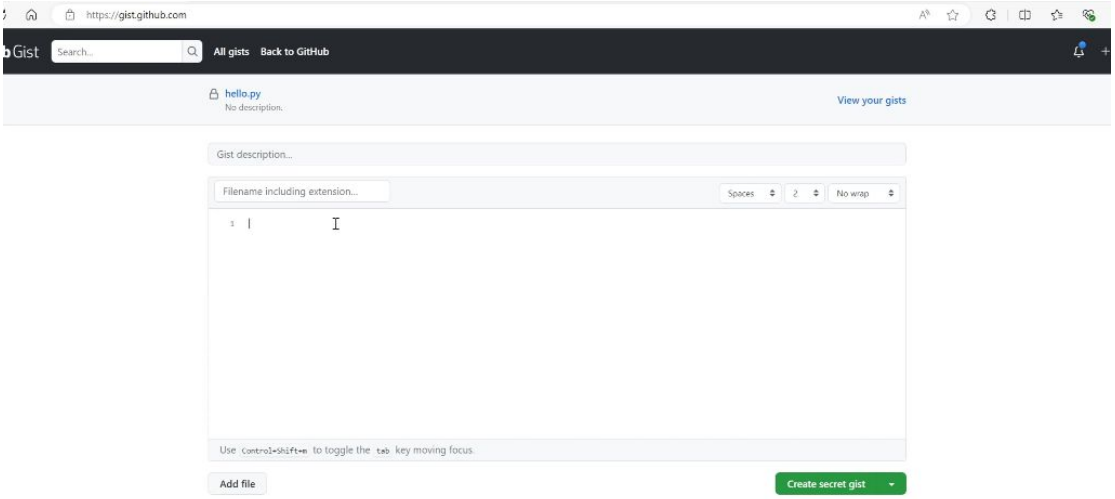
8）上传文件 06:16




- 
- **操作步骤**：

    - 点击"Add file"→"Upload files"
    - 选择本地文件
    - 填写commit message
    - 提交变更
- **注意事项**：每次上传都会生成新的commit记录

9）GitHub Gist 06:43




Github gist一般用来分享单个代码文件

- 
- **用途**：分享代码片段或单个文件
- **分享方式**：
    - **嵌入HTML**：通过
    - **分享链接**：直接发送gist链接
    - **克隆支持**：支持HTTPS和SSH方式克隆
- **权限控制**：可创建公开或私密gist

3. 内容总结 08:13

- **核心流程**：创建仓库→选择协议→编辑文件→提交变更
- **关键概念**：
    - 开源协议的选择标准

- commit的版本控制机制
- Gist的轻量级代码分享功能

- **操作技巧**：
  - 使用短ID引用commit
  - 善用Markdown编写README
  - 通过嵌入代码分享技术片段

*二、知识小结*

| 知识点 | 核心内容 | 考试重点/易混淆点 | 难度系数 |
| --- | --- | --- | --- |
| **GitHub 仓库创建** | 通过右上角加号创建新存储库，填写名称、描述，选择公开/私有，添加 README 文件 | 公开与私有仓库的区别（**私有仓库现免费**） | ★★ |
| **开源协议选择** | **GPL 系**（衍生作品必须开源，如 Linux） vs **宽松协议**（MIT/BSD，可闭源） | **GPL 传染性**vs MIT 仅需保留协议副本 | ★★★ |
| **README 文件编辑** | 使用 Markdown 语法编写项目说明，支持预览与提交（commit） | commit是 Git 快照记录，形成可回溯历史链路 | ★★ |
| **文件上传与版本管理** | 通过网页上传文件，每次提交生成唯一 **Commit ID**（短 ID 与长 SHA 值） | Commit ID 作用及查看历史状态的方法 | ★★★ |
| **GitHub Gist** | 轻量级代码片段仓库，支持嵌入 HTML 或通过链接分享（如 Redis 分布式锁示例） | Gist 的 Git 操作（克隆通过 HTTPS/SSH） | ★★ |